# AutoTuneAI 使用說明

AutoTuneAI 目前是一個「包住訓練或 benchmark 入口」的資源監控、限制、分析與 runtime tuning 工具。它的設計目標不是取代你的訓練環境，而是讓 AutoTuneAI 跑在自己的環境中，再去啟動另一個 Python、conda、venv、shell script 或 Docker container 內的 training command。

## 1. 安裝與基本檢查

建議先在 WSL/Linux 裡建立 AutoTuneAI 自己的環境：

```
conda env create -f environment.yml
conda activate autotuneai
python -m pip install -e .
```

如果環境已存在：

```
conda activate autotuneai
python -m pip install -r requirements-core.txt
python -m pip install -e .
```

確認 CLI 可用：

```
autotuneai --help
python -m autotune.cli --help
```

直接驗證 repo 內建 demo：

```
autotuneai demo
autotuneai demo --scenario tune-batch
```

跑 repo 內建真實訓練 workload：

```
autotuneai run -- python examples/iris_train.py --config
examples/iris_train_config.yaml
autotuneai run --memory-budget-gb 1.5 --hard-kill -- python
examples/stress_train.py --config examples/stress_train_config.yaml
```

跑完整測試：

```
python -m unittest discover -s tests
```

這個指令會從 `tests/` 目錄自動尋找 `test_*.py`，然後執行全部 unittest。成功時最後會看到`OK`。

## 2. 不要用 sudo su 進 root shell 跑 AutoTuneAI

正確做法是讓 AutoTuneAI 留在使用者的 conda 環境，需要 root 權限時只透過 `--sudo` 或 `sudo -v` 讓特定 systemd/sysctl 動作用 root 執行。

```
sudo -v
autotuneai run --executor systemd --sudo --memory-budget-gb 22 -- python train.py
```

不要這樣做：

```
sudo su
conda activate autotuneai
```

原因是 root shell 通常沒有使用者的 conda 初始化，會發生 `conda: command not found` 或指到錯的 Python。AutoTuneAI 只需要 systemd/cgroup/sysctl 這些動作有權限，訓練程式本身不應該因此變成 root 身份執行。

## 3. 單一 CLI 入口

主要指令：

```
autotuneai inspect
autotuneai executors
autotuneai run -- <command>
autotuneai analyze --run-id <run_id>
autotuneai report --run-id <run_id>
autotuneai calibrate-memory --budget-gb -5 -3 --workload-memory-mb 2048
autotuneai tune-system
autotuneai tune-system --apply --sudo
autotuneai tune-batch --file train.yaml --key batch_size --values 64 32 16 --
python train.py
autotuneai list-runs
autotuneai restore --run-id <run_id> --sudo
```

舊 script 入口仍保留，例如：

```
python scripts/run_with_budget.py -- python train.py
python scripts/tune_training_config.py --file train.yaml --values 64 32 -- python
train.py
```

但新使用者建議優先使用 `autotuneai`。

## 4. Executor 類型

查看本機可用能力：

```
autotuneai executors --probe-systemd --probe-docker --check-sudo-cache
```

目前 executor：

```
local
  跨平台 fallback。使用 psutil 做 process-tree 監控。
  CPU affinity 在 Linux 可用；memory limit 是軟限制，搭配 --hard-kill 才會主動終止。

systemd
  Linux/WSL2 + systemd 可用。透過 transient scope + cgroup 做硬 memory/CPU limit。
  常需要 sudo，因此建議先 sudo -v，再加 --sudo。

docker
  透過 docker run --memory/--cpus 做跨平台 container 限制。
  使用者需要準備含有訓練相依套件的 Docker image。

windows_job
  目前只做 capability 規劃，尚未實作。

macos
  目前走 local 或 Docker；macOS 原生 hard memory executor 尚未實作。
```

自動選擇：

```
autotuneai run --executor auto --allow-sudo-auto --memory-budget-gb 22 -- python
train.py
```

如果 systemd 需要 sudo 而你沒有加 `--allow-sudo-auto`，AutoTuneAI 會停止並提示你明確 opt in。

## 5. 包住另一個訓練環境

AutoTuneAI 不要求使用者把訓練相依套件裝到 AutoTuneAI 的環境。你可以從 AutoTuneAI 環境啟動另一個環境的 Python。

直接指定另一個 venv/conda 的 Python：

```
autotuneai run \
  --executor auto \
  --memory-budget-gb 22 \
```

```
  --reserve-cores 1 \
  --cpu-quota-percent 90 \
  -- /path/to/user/env/bin/python train.py --config configs/train.yaml
```

使用 `conda run`：

```
autotuneai run \
  --memory-budget-gb 22 \
  -- conda run -n user-train-env python train.py --config configs/train.yaml
```

在 WSL 裡用完整 conda 路徑更穩：

```
/home/louis/miniforge3/bin/conda run -n autotuneai autotuneai run \
  --executor systemd \
  --sudo \
  --memory-budget-gb 22 \
  -- /home/louis/miniforge3/bin/conda run -n user-train-env python train.py
```

這就是「雙環境」模型：AutoTuneAI 的環境只負責監控、限制、記錄和調度；使用者原本的 training environment 負責真正訓練。

## 6. Docker 雙環境用法

如果你要讓別人 clone 後更容易跑，Docker 是最乾淨的方式之一。AutoTuneAI 只需要本機有 Docker CLI/daemon，訓練相依套件放在 image 裡。

```
autotuneai run \
  --executor docker \
  --docker-image my-training-image:latest \
  --memory-budget-gb 22 \
  --cpu-quota-percent 90 \
  -- python train.py --config configs/train.yaml
```

Docker executor 會把目前 repo 掛到 container 的 `/workspace`，並在 `/workspace` 內執行命令。

重要限制：

- `python:3.12-slim` 是預設 image，只適合簡單 Python script。
- 真正訓練 PyTorch/Transformers/資料集時，請建立自己的 Docker image。
- Docker executor 目前主要負責硬限制；host process sampling 看到的是 docker client，不等於 container 內所有細節。後續可再補 `docker stats` 或 cgroup path 解析。

## 7. Memory budget

正數代表絕對 GB 上限：

```
autotuneai run --memory-budget-gb 22 -- python train.py
```

負數代表「距離 Linux/WSL 可見總記憶體跑滿，至少保留多少 GB」：

```
autotuneai run --memory-budget-gb -5 -- python train.py
```

如果 WSL 可見總記憶體是 23.7GB，`-5` 會轉成約 18.7GB 的 effective budget。實際 Windows Task Manager 看到的剩餘值可能不同，因為有 page cache、shared memory、Python allocator、WSL memory reclamation 等因素。

校準本機 memory 行為：

```
autotuneai calibrate-memory \
  --budget-gb -5 -3 1 \
  --workload-memory-mb 2048 \
  --duration-seconds 10 \
  --workers 4 \
  --sample-interval-seconds 0.1
```

輸出：

```
results/reports/memory_calibration.json
```

裡面會記錄每個 budget 的：

```
effective_budget_mb
peak_memory_mb
observed_min_available_memory_gb
reserve_error_gb
memory_budget_exceeded
run_id
```

如果你要硬限制，Linux/WSL 建議：

```
sudo -v
autotuneai run --executor systemd --sudo --memory-budget-gb 22 -- python train.py
```

Docker 建議：

```
autotuneai run --executor docker --docker-image my-training-image:latest --memory-budget-gb 22 -- python train.py
```

## 8. CPU 限制與保留核心

保留 1 個 logical core：

```
autotuneai run --reserve-cores 1 -- python train.py
```

限制總 CPU 約 90%：

```
autotuneai run --cpu-quota-percent 90 -- python train.py
```

兩者一起用時，AutoTuneAI 會取更嚴格的 allowed thread 數。8 logical cores 下：

```
--reserve-cores 1     => allowed_threads = 7
--cpu-quota-percent 50 => allowed_threads = 4
```

local executor 在 Linux 主要靠 CPU affinity，所以 Windows Task Manager/WSL host UI 可能看到每個 vCPU 都有一些活動，不一定會呈現「某一顆 core 完全 0%」。判斷是否生效請看：

```
autotuneai analyze --run-id <run_id>
```

重點欄位：

```
allowed_threads
affinity_cores
expected_max_total_cpu_percent
observed_peak_process_cpu_percent
```

如果要更硬的 CPU cap，優先用：

```
autotuneai run --executor systemd --sudo --cpu-quota-percent 90 -- python train.py
autotuneai run --executor docker --cpu-quota-percent 90 --docker-image my-image --python train.py
```

## 9. 高壓測試命令

CPU 壓滿，但保留 1 core：

```
autotuneai run \
  --executor local \
  --reserve-cores 1 \
  --sample-interval-seconds 0.1 \
  -- python scripts/stress_workload.py --workers 8 --duration-seconds 60 --memory-mb 256
```

CPU 限制 90%：

```
autotuneai run \
  --executor local \
  --cpu-quota-percent 90 \
  --sample-interval-seconds 0.1 \
  -- python scripts/stress_workload.py --workers 8 --duration-seconds 60 --memory-mb 256
```

Memory soft guard 測試：

```
autotuneai run \
  --executor local \
  --memory-budget-gb 0.5 \
  --hard-kill \
  --sample-interval-seconds 0.05 \
  -- python scripts/stress_workload.py --workers 2 --duration-seconds 30 --memory-mb 2048
```

Systemd hard memory/CPU 測試：

```
sudo -v
autotuneai run \
  --executor systemd \
  --sudo \
  --memory-budget-gb 0.5 \
  --cpu-quota-percent 90 \
  --sample-interval-seconds 0.1 \
  -- python scripts/stress_workload.py --workers 8 --duration-seconds 60 --memory-mb 2048
```

## 10. Runtime system tuning

查看建議，不修改系統：

```
autotuneai tune-system
```

套用 Linux/WSL training-safe profile：

```
sudo -v
autotuneai tune-system --apply --sudo
```

Linux/WSL profile 會嘗試調整 runtime sysctl/sysfs，例如：

```
vm.swappiness
kernel.numa_balancing
vm.dirty_background_ratio
vm.dirty_ratio
vm.zone_reclaim_mode
```

每次套用都會建立 run directory：

```
.autotuneai/runs/<run_id>/
  system_tuning_plan.json
  system_tuning_before.json
  system_tuning_after.json
  system_tuning_diff.json
```

恢復：

```
autotuneai restore --run-id <run_id> --sudo
```

也可以把 system tuning 包進訓練 lifecycle，訓練完自動恢復：

```
sudo -v
autotuneai run \
  --tune-system linux-training-safe \
  --system-tuning-sudo \
  --memory-budget-gb 22 \
  -- python train.py
```

Windows profile 目前走保守可還原的 `powercfg` active power scheme 調整。套用時會先記錄目前電源方案 GUID，tuned run 結束後用 snapshot 還原：

```
autotuneai tune-system --profile windows-throughput
autotuneai tune-system --profile windows-throughput --apply
autotuneai restore --run-id <run_id>
```

也可以改用自動模式，讓 AutoTuneAI 依目前平台選建議的 runtime system tuning profile：

```
sudo -v
autotuneai run \
  --auto-tune-system \
  --system-tuning-sudo \
  --executor systemd \
  --sudo \
  --memory-budget-gb 22 \
  -- python train.py
```

只要 run 有套用 system tuning， run directory 會包含：

```
system_tuning_before.json
system_tuning_after.json
system_tuning_diff.json
system_tuning_restore_after.json
```

目前可用 profile：

```
linux-training-safe
  一般訓練用的保守 profile。
  包含 swappiness、dirty page ratio、zone reclaim、NUMA balancing、Transparent Huge
Pages madvise。

linux-memory-conservative
  RAM 緊張時使用，會更積極保留訓練記憶體空間。
  包含 swappiness=1、vfs_cache_pressure=200、page-cluster=0、較低 dirty ratio、THP
madvise。

linux-throughput
  資料讀取或 checkpoint throughput 優先時使用。
  包含較高 dirty ratio、較低 vfs_cache_pressure、THP madvise。

linux-low-latency
  想降低 flush/THP latency spike 時使用。
  包含較低 dirty ratio、較頻繁 writeback、THP never。

linux-cpu-conservative
  CPU quota 或 reserve cores 場景使用，減少 kernel 背景 flush 對 workload CPU 的干
擾。
```

```
windows-training-safe
  Windows 一般訓練 profile，暫時切到 High performance power scheme，結束後還原。

windows-memory-conservative
  Windows 沒有 Linux sysctl 類型的安全記憶體 runtime knob；目前搭配 AutoTuneAI
memory budget 監控與 High performance power scheme。

windows-throughput
  Windows throughput profile，降低 CPU downclocking 對訓練吞吐的干擾。

windows-low-latency
  Windows latency profile，降低 CPU frequency ramp-up 延遲。

windows-cpu-conservative
  Windows CPU/thermal 保守 profile，暫時切到 Balanced power scheme。
```

自動選擇規則：

```
--auto-tune-system + 有 memory budget / reserve memory
  -> Linux/WSL: linux-memory-conservative
  -> Windows: windows-memory-conservative

--auto-tune-system + 沒有 memory budget
  -> Linux/WSL: linux-training-safe
  -> Windows: windows-training-safe

--auto-tune-system + 有 cpu quota / reserve cores
  -> Linux/WSL: linux-cpu-conservative
  -> Windows: windows-cpu-conservative
```

手動指定更激進的 profile：

```
sudo -v
autotuneai run \
  --tune-system linux-low-latency \
  --system-tuning-sudo \
  --executor systemd \
  --sudo \
  --memory-budget-gb -3 \
  -- python train.py
```

如果你想知道調教到底有沒有讓同一個 workload 變好，使用 A/B comparison：

如果你只是先驗證這個 repo，請直接用內建的 `examples/dummy_train.py`。`train.py` 和 `configs/train.yaml` 在這裡代表你自己的訓練程式與設定檔，不是 repo 內建檔案。

```
sudo -v
autotuneai compare-tuning \
  --workload-profile memory \
  --executor systemd \
  --sudo \
  --system-tuning-sudo \
  --memory-budget-gb -3 \
  --sample-interval-seconds 0.1 \
  --repeat 3 \
  -- python examples/dummy_train.py
```

要看 system tuning 是否真的對訓練有效，不建議用太短的 dummy workload。正式 benchmark 請用至少 60 秒的 stress workload：

```
sudo -v
autotuneai compare-tuning \
  --workload-profile memory \
  --executor systemd \
  --sudo \
  --system-tuning-sudo \
  --memory-budget-gb -3 \
  --sample-interval-seconds 0.1 \
  --repeat 3 \
  -- python examples/stress_train.py --config
examples/stress_train_60s_config.yaml
```

如果要測 memory pressure，用較接近滿載的設定，並搭配 `--memory-budget-gb -3` 或更保守的 reserve：

```
sudo -v
autotuneai compare-tuning \
  --workload-profile memory \
  --executor systemd \
  --sudo \
  --system-tuning-sudo \
  --memory-budget-gb -3 \
  --sample-interval-seconds 0.1 \
  --repeat 3 \
  -- python examples/stress_train.py --config
examples/stress_train_memory_pressure_config.yaml
```

Windows 上不使用 `systemd` / `sudo`，直接用 local executor；tuned run 會套用 Windows profile，結束後還原 power scheme：

```
autotuneai compare-tuning `
  --workload-profile throughput `
  --executor local `
```

```
  --sample-interval-seconds 0.1 `
  --repeat 3 `
  -- python examples/stress_train.py --config
examples/stress_train_60s_config.yaml
```

這會先跑 baseline，再跑 tuned，最後輸出：

```
results/reports/tuning_comparison.json
```

重要欄位：

```
baseline.run_id
tuned.run_id
deltas.lifecycle_duration_seconds
deltas.lifecycle_duration_percent
deltas.adjusted_lifecycle_duration_seconds
deltas.adjusted_lifecycle_duration_percent
deltas.benchmark_duration_seconds
deltas.benchmark_duration_percent
deltas.workload_duration_seconds
deltas.workload_duration_percent
deltas.system_tuning_overhead_seconds
deltas.workload.samples_per_second.percent
deltas.peak_memory_mb
deltas.peak_memory_percent
deltas.min_available_memory_mb
```

判讀時要分清楚：

- `lifecycle_duration_*` 是整段 AutoTuneAI run 時間，包含 system tuning apply/restore。
- `system_tuning_overhead_seconds` 是 system tuning apply + restore 的成本。
- `adjusted_lifecycle_duration_*` 是 `lifecycle_duration_*` 扣掉 system tuning apply/restore 後的時間，適合沒有 workload metrics 的通用 command。
- `workload_duration_*` 是 workload 自己寫出的訓練時間，若有提供會優先作為 benchmark duration。
- `benchmark_duration_*` 是公平比較用時間；有 workload metrics 時等於 workload duration，否則等於 adjusted lifecycle duration。
- `deltas.workload.samples_per_second.percent` 是訓練吞吐變化，正值代表 tuned workload 更快。
- `workload` 區塊只保留 performance metrics；不輸出 accuracy、loss、dice 這類任務品質指標，避免把 tuning comparison 綁死在分類任務。
- `peak_memory_percent` 負值代表 tuned run peak memory 更低。

短 workload 很容易被 system tuning apply/restore overhead 或排程雜訊蓋掉，所以正式比較請用 `--repeat 3` 以上，並看 aggregate median。

## 10.1 NVIDIA GPU runtime tuning

如果機器有 NVIDIA GPU 且 `nvidia-smi` 在 PATH，可以檢查 GPU tuning plan：

```
autotuneai tune-gpu
```

目前 profile：

```
nvidia-safe
  開 persistence mode，不主動調 power limit。

nvidia-throughput
  開 persistence mode，並嘗試把 power limit 設到 nvidia-smi 回報的 max limit。
```

套用 GPU runtime tuning：

```
sudo -v
autotuneai tune-gpu --apply --sudo --profile nvidia-throughput
```

輸出會包含：

```
gpu_tuning_before.json
gpu_tuning_after.json
gpu_tuning_diff.json
```

恢復：

```
autotuneai restore --run-id <run_id> --gpu-sudo
```

也可以把 GPU tuning 包進訓練 lifecycle：

```
sudo -v
autotuneai run \
  --auto-tune-system \
  --auto-tune-gpu \
  --system-tuning-sudo \
  --gpu-tuning-sudo \
  --executor systemd \
  --sudo \
  --memory-budget-gb -3 \
  -- python train.py
```

限制：

- `nvidia-smi` 某些欄位在 laptop GPU 或 WSL 可能是 `[N/A]`，AutoTuneAI 會略過無法套用的 setting。
- power limit / persistence mode 可能需要 root/admin 權限，也可能被 OEM/driver 鎖住。
- 目前還沒有自動調 CUDA allocator、MIG、application clocks；那些會是下一層 GPU tuner。

`autotuneai report --run-id <run_id>` 的 `Before / After` 區塊會集中顯示 memory start/end/min、peak memory、system tuning snapshots，以及 source/config change 數量。

如果被中斷，仍可用 run id 做 restore：

```
autotuneai list-runs
autotuneai restore --run-id <run_id> --sudo
```

BIOS/UEFI、SMT、Turbo Boost、fan curve、XMP/EXPO 這些不是 runtime tool 能安全跨機器修改的範圍。目前 AutoTuneAI 聚焦 runtime-level tuning，原因是可記錄、可回復、可跨機器。

## 11. Reversible source/config tuning

單次 source edit dry run：

```
python scripts/tune_source.py \
  --file train.py \
  --find "batch_size = 64" \
  --replace "batch_size = 32"
```

真的套用：

```
python scripts/tune_source.py \
  --file train.py \
  --find "batch_size = 64" \
  --replace "batch_size = 32" \
  --apply
```

套用後會在 run manifest 記錄修改前內容。恢復：

```
autotuneai restore --run-id <run_id>
```

包住訓練，跑完自動恢復：

```
python scripts/run_tuned_with_budget.py \
  --edit train.py "batch_size = 64" "batch_size = 32" \
  --memory-budget-gb 22 \
  -- python train.py
```

不要自動恢復：

```
python scripts/run_tuned_with_budget.py \
  --edit train.py "batch_size = 64" "batch_size = 32" \
  --keep-changes \
  -- python train.py
```

## 12. Training config tuner

AutoTuneAI 可以調任意「單一 numeric key」，不是只能調 batch size。

範例 YAML：

```
batch_size: 64
num_workers: 4
gradient_accumulation_steps: 1
```

調 batch size：

```
autotuneai tune-batch \
  --file examples/train_config.yaml \
  --key batch_size \
  --values 128 64 32 16 \
  --memory-budget-gb 22 \
  --executor auto \
  -- python examples/dummy_train.py
```

如果你要一次調多個 numeric training knobs，用 `tune-training`：

```
autotuneai tune-training \
  --file examples/iris_train_config.yaml \
  --knob batch_size=8,16,32 \
  --knob gradient_accumulation_steps=1,2,4 \
  --knob preload_copies=4,8,12 \
  --objective throughput \
  -- python examples/iris_train.py --config examples/iris_train_config.yaml
```

調 dataloader workers：

```
autotuneai tune-batch \
  --file examples/train_config.yaml \
  --key dataloader_workers \
  --values 0 2 4 8 \
  --memory-budget-gb 22 \
```

```
  --reserve-cores 1 \
  -- python examples/dummy_train.py
```

調 gradient accumulation：

```
autotuneai tune-batch \
  --file configs/train.yaml \
  --key gradient_accumulation_steps \
  --values 1 2 4 \
  -- python train.py --config configs/train.yaml
```

支援的 assignment 格式：

```
batch_size: 64
```

```
batch_size = 64
```

限制：

- 同一個 key 在檔案中只能出現一次，避免改錯。
- 目前只支援整數 numeric value。
- 每個 trial 都會先改 config、跑訓練、記錄 run、再恢復原檔。
- summary 預設輸出到 `results/reports/training_tuning_summary.json`。

summary 重要欄位：

```
key
original_value
candidate_values
recommended_value
recommended_run_id
trials[].value
trials[].safe
trials[].reason
trials[].resource_summary
```

## 13. Run artifacts、分析與報告

每次 run 會建立：

```
.autotuneai/runs/<run_id>/
  manifest.json
```

```
  env.json
  before_status.txt
  before_diff.patch
  head.txt
  resource_timeline.json
  resource_summary.json
```

列出 run：

```
autotuneai list-runs
```

分析 run：

```
autotuneai analyze --run-id <run_id>
autotuneai analyze --run-id <run_id> --json
```

產生 Markdown 報告：

```
autotuneai report --run-id <run_id>
```

預設輸出：

```
.autotuneai/runs/<run_id>/report.md
```

報告會包含：

```
Summary
Executor
CPU
Memory
Cgroup
Diagnostics
System Tuning Diff
```

## 14. 專案最終目標

最終版希望長成：

```
使用者 clone repo
建立 AutoTuneAI 環境
```

```
autotuneai inspect / executors 看本機能力
autotuneai run 包住自己的 training command
autotuneai tune-batch / tune-system / calibrate-memory 做調教
autotuneai analyze / report 看 before-after 與效果
autotuneai restore 在中斷或失敗後恢復 runtime/source/config
```

希望能支援：

- WSL/Linux systemd hard limits
- Docker cross-platform hard limits
- Windows Job Object native executor
- macOS fallback + Docker
- training config tuning
- reversible source tuning
- runtime sysctl tuning with before/after/diff/restore
- run report 與校準報告

## 15. 市面上類似工具

相近但不完全相同的工具：

```
systemd-run / cgroups
  能限制資源，但不懂 training workflow、config tuning、報告與恢復。

Docker
  能做 container limit，但不會自動分析 training budget，也不會幫你改 config 後恢復。

Kubernetes / Slurm / Ray
  適合 cluster scheduling，但對單機 WSL/個人工作站太重。

Weights & Biases / MLflow
  擅長 experiment tracking，不是 system/runtime resource guard。

Optuna / Ray Tune
  擅長 hyperparameter tuning，但不是專門做 root/system/cgroup/runtime guard。

PyTorch Profiler
  擅長 profiling，不負責 system tuning、cgroup hard limit 或 source restore。
```

AutoTuneAI 的定位是把「單機訓練入口包住」，同時做資源限制、runtime tuning、config tuning、分析報告與恢復。

## 16. 目前限制

- BIOS/UEFI 不做自動修改，只能提供人工建議。
- local executor 的 memory limit 是監控加 hard-kill，不是 kernel hard cap。
- systemd executor 需要 Linux/WSL2 + systemd，且可能需要 sudo。
- Docker executor 需要使用者提供合適 image。

- Windows Job Object executor 尚未實作。
- config tuner 目前只支援單一檔案、單一 numeric key。